Lecture 1: Classification

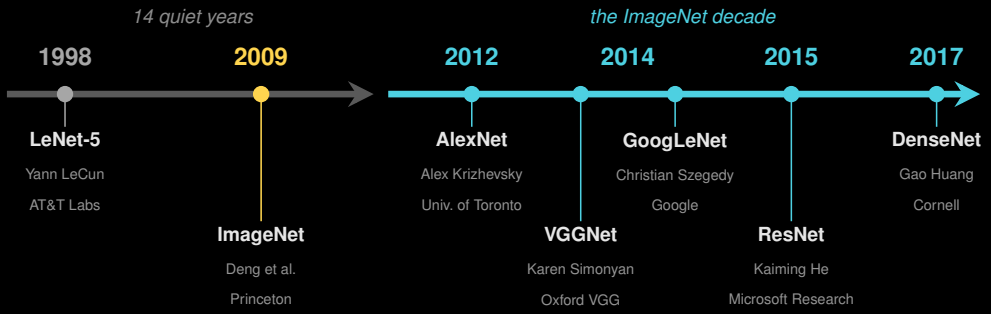
A short history: from LeNet to ResNet

Mehmet Kurt, Tianyi Ren
University of Washington

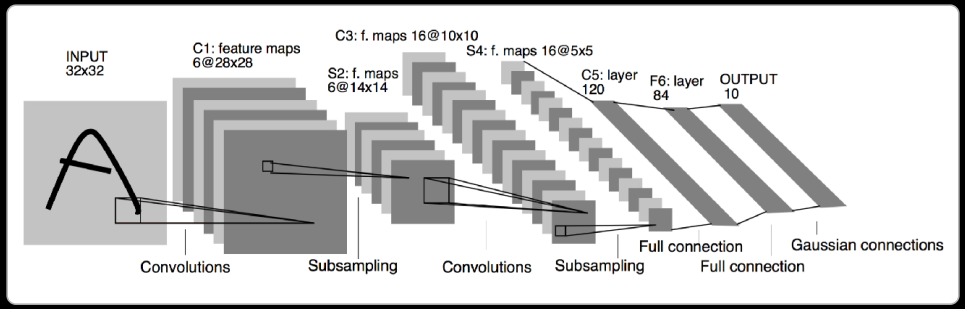
August 30, 2026

A short history of deep image classification

TWENTY YEARS, SIX ARCHITECTURES



LeNet-5 (LeCun et al., 1998)



The pioneering work

- **Automatic feature extraction:** the convolutional layers learned filters to detect edges, corners, and patterns automatically.
- **Hierarchical representation:** it processed data in layers, from simple visual patterns to complex digit shapes.
- An MLP on raw pixels is **intractable** and wasteful, since pixels are **spatially correlated**. **CNNs** extract **meaningful, low-dimensional features** instead.
- **Practical success:** remarkable accuracy on digit recognition, proving CNNs could outperform traditional ML models.

LeCun et al., *Gradient-Based Learning Applied to Document Recognition*, Proc. IEEE

LeNet-5 results: MNIST

60,000 original images

Test error: **0.95%**

MNIST handwritten digits.

540,000 artificial distortions + 60,000 original

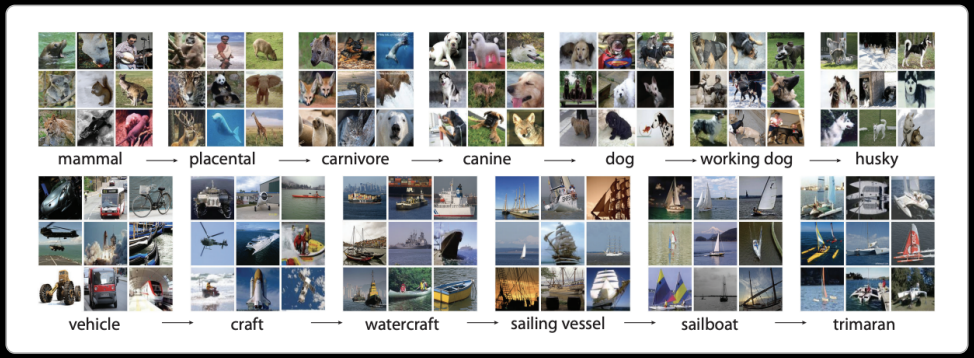
Test error: **0.8%**

The 82 misclassified test digits.

Distorting scarce data to get more of it worked already in 1998; Lecture 2 returns to this trick when medical labels get scarce.

adapted from A. Singh, CMU-10701

ImageNet fixed the data side (2009)



Each row walks down the WordNet noun hierarchy, coarse to fine. ImageNet holds images at *every* node, not only at the leaves.

The scale, and the labels

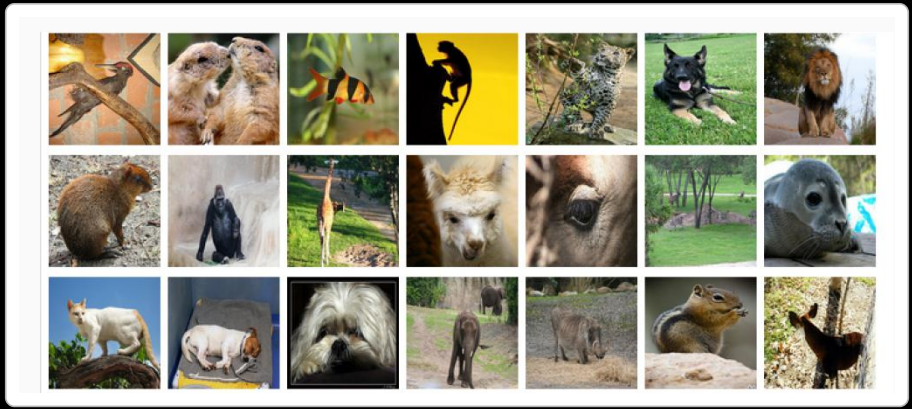
- **1.2M** images, **1000** classes, against 60k for CIFAR-10.
- Classes are **WordNet nouns**, crowdsourced rather than expert-annotated. Scale came from *cheap* labels.
- **ILSVRC**: an annual competition on a sealed test set, so every method was scored the same way.

Deng et al., *ImageNet*, CVPR

IMAGENET: THE CLASSIFICATION TASK

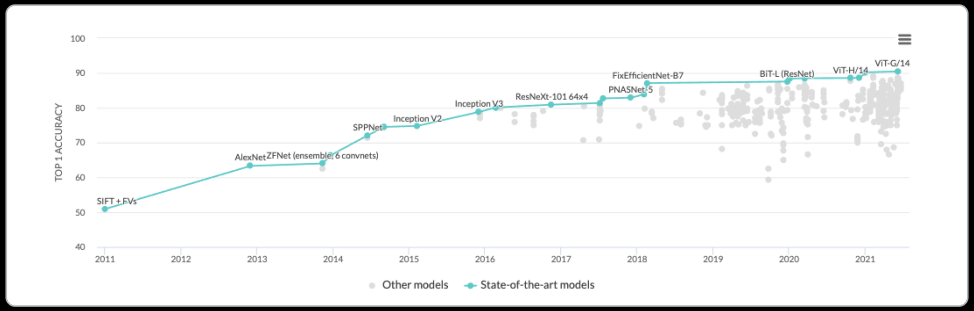
Classification goals

- Make **1 guess** about the label: **top-1 error**.
- Make **5 guesses** about the label: **top-5 error**.



adapted from A. Singh, CMU-10701

IMAGENET BECAME THE SCOREBOARD

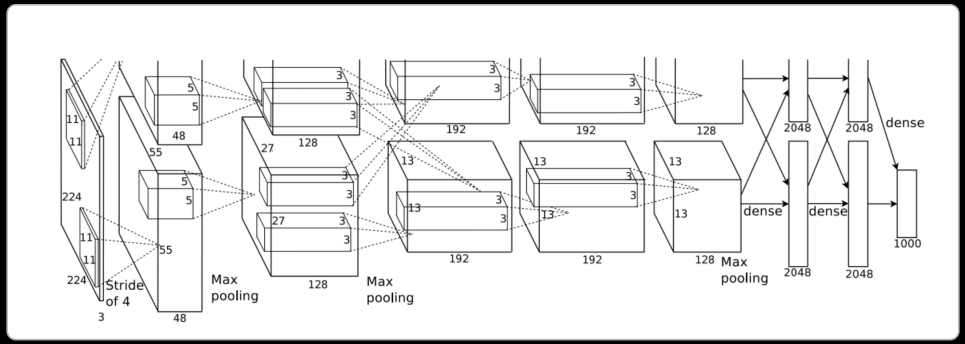


Classical methods had hit a performance plateau. In 2012, Krizhevsky et al. took first place with their CNN-based network, **AlexNet**. Since then, the leaderboard has consisted of deep networks: first CNNs, and since 2020 vision transformers (the top points on this chart).

Axis note: this chart is top-1 *accuracy* (higher is better); the rest of this lecture quotes top-5 *error* (lower is better).

Papers with Code, *Image Classification on ImageNet*

AlexNet (2012): the result that restarted the field



5 convolutional + 3 fully connected layers, split across two GPUs (the two rows).

At a glance

- **8 layers**: input 224×224×3, 5 convolutional + 3 fully connected. About **60M** parameters.
- **Block:** conv → **ReLU** → max pool, with dropout in the FC stack. Split across two GPUs because it did not fit on one.
- **Result: 16.4%** top-5 error against **26.2%** for the best non-CNN entry, a ten-point jump in a contest that usually moved by one or two.

Krizhevsky, Sutskever & Hinton, *ImageNet Classification with Deep CNNs*, NeurIPS

What AlexNet actually changed

AlexNet keeps the same approach: successive filters designed to capture more and more subtle features. But the work explored several **crucial details**.

1. ReLU

Better nonlinearity: the derivative is 0 if the feature is below 0 and 1 for positive values. Efficient for **gradient propagation**.

2. Dropout

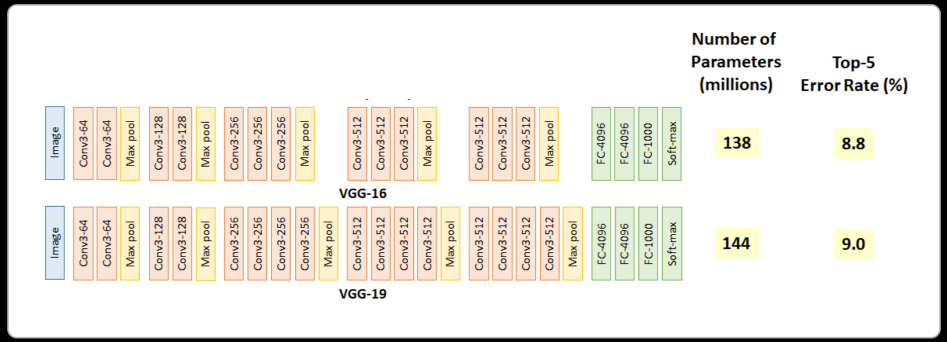
Regularization: force the network to **forget things at random**, so that it sees the next input from a better perspective.

3. Augmentation

Images are shown with random **translation, rotation, crop**: the network learns the **attributes** of the images, rather than the images themselves.

Amidi & Amidi, *The evolution of image classification explained*

VGGNet (2014): deep is better



The two configurations everyone still uses. Full list in the paper.

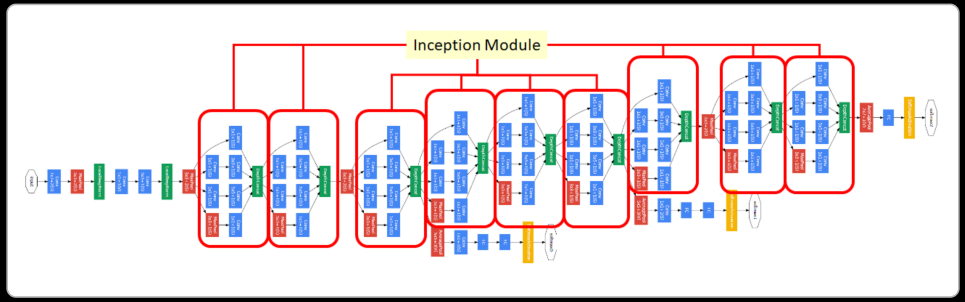
Why small filters? Three reasons

1. Small filters induce **more nonlinearity**, which means more degrees of freedom for the network.
2. Stacking them lets the network **see more than it looks like**: two 3×3 layers see a 5×5 receptive field, three see 7×7. The same feature extraction as with large filters.
3. Small filters also **limit the number of parameters**, which is good when you want to go that deep.

Quantitatively: **7.3% top-5 error** on ImageNet.

Simonyan & Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR

GoogLeNet (2014): time for inception



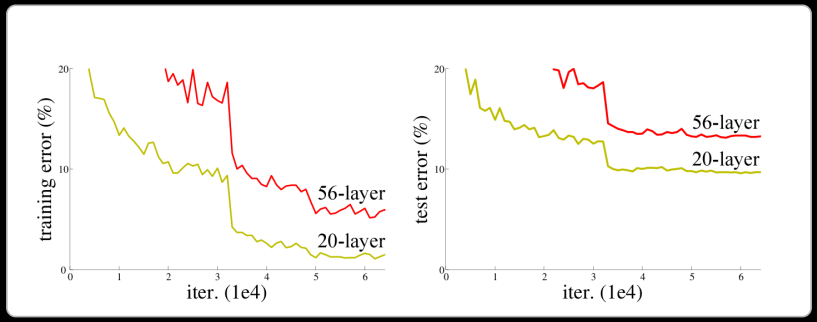
Nine stacked Inception modules (outlined in red), two auxiliary classifiers, and no large FC stack at the end.

Inception modules

- Convolutions with **different filter sizes** are processed on the same input, then **concatenated**: multilevel feature extraction at each step. General features from the 5×5 filters, more local features from the 3×3 ones.
- Insanely expensive to compute? The brilliant solution: 1×1 **convolutions**. They reduce the dimensionality of the features, and combine feature maps in a way that helps the representation.
- Why "inception"? The modules are **networks stacked inside a bigger network**. The best GoogLeNet ensemble: **6.7% top-5 error** on ImageNet.

Szegedy et al., *Going Deeper with Convolutions*, CVPR

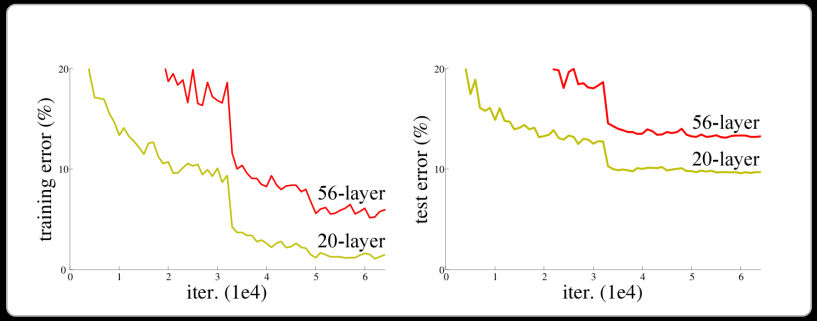
AND THEN DEPTH STOPPED HELPING



Ask the room: what is wrong with these curves? Look at *both* panels before answering.

He et al., *Deep Residual Learning for Image Recognition*, CVPR

AND THEN DEPTH STOPPED HELPING



The degradation problem

With the network depth increasing, accuracy gets **saturated** (which might be unsurprising) and then **degrades rapidly**. Unexpectedly, such degradation is **not caused by overfitting**: adding more layers to a suitably deep model leads to **higher training error**. The degradation of training accuracy indicates that **not all systems are similarly easy to optimize**.

He et al., *Deep Residual Learning for Image Recognition*, CVPR

ResNet: making depth trainable

THE ARGUMENT THAT GIVES AWAY THE ANSWER

Hypothesis: the problem is an **optimization** problem; deeper models are harder to optimize.

In principle

The deeper model should be able to perform **at least as well** as the shallower model.

A solution by construction

Copy the **learned layers** from the shallower model, and set the additional layers to **identity mapping**.

learn $H(x)$ from scratch
plain layer

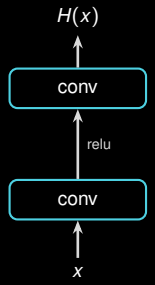
learn $\mathcal{F}(x) = H(x) - x$
residual layer

If the best thing to do is nothing, the plain layer must *build* the identity; the residual layer just sets $\mathcal{F} \rightarrow 0$.

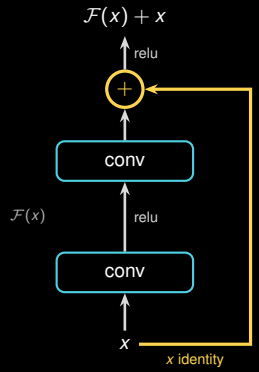
He et al., CVPR. adapted from CS231n

The Residual Block

Structure each layer to fit a **residual function** with respect to the identity, then add the two back together.



"Plain" layers



Residual block

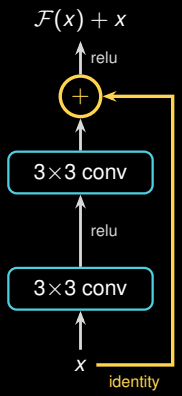
Residual learning: why this works

Every two layers, an **identity mapping** joins the output via **elementwise addition**. This is very helpful for **gradient propagation**: the error can be backpropagated through **multiple paths**. From a representation point of view, it also combines **different levels of features** at each step, just as the inception modules did. And it costs **no extra parameters**; it is an addition.

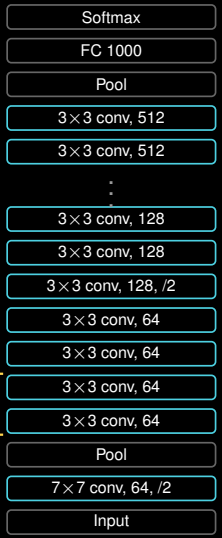
ResNet: a stack of residual blocks

Connect the layers

- All the networks so far followed the same trend: **going deeper**. But stacking more layers does not lead to better performance; the exact opposite occurs.
- Why? In one word: **the gradient**. The trick that counters it: **residual learning**.
- Every two layers, an **identity mapping** joins in via elementwise addition; the block on the left, stacked **18 to 152 layers** deep.
- To this date one of the best performing networks on ImageNet, with a **3.6% top-5 error rate**.

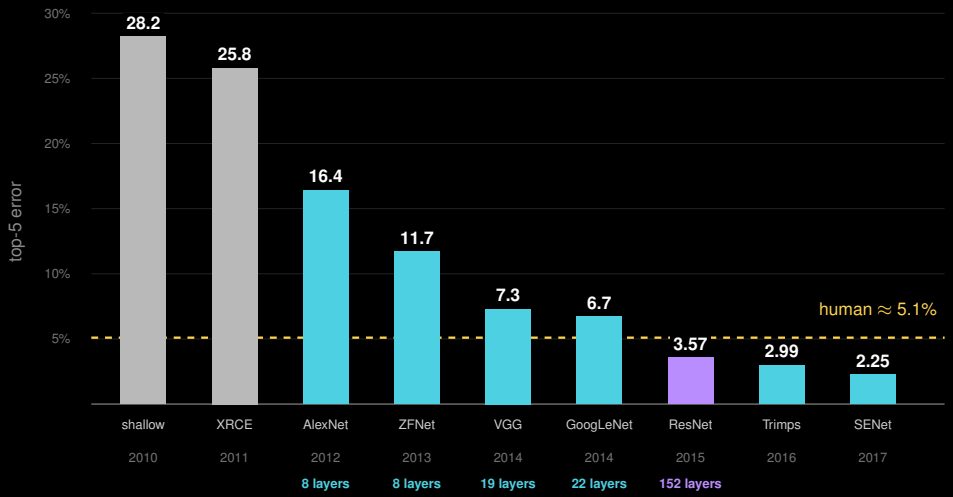


Residual block



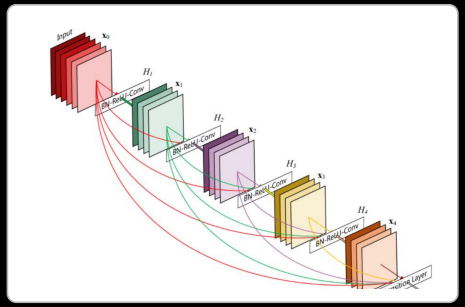
He et al., *Deep Residual Learning for Image Recognition*, CVPR. adapted from CS231n

THE REVOLUTION OF DEPTH



Error fell 28% → 2% in seven years, and the depth column tells you how: **8** → **19** → **22** → **152 layers**, once the residual connection made 152 trainable.

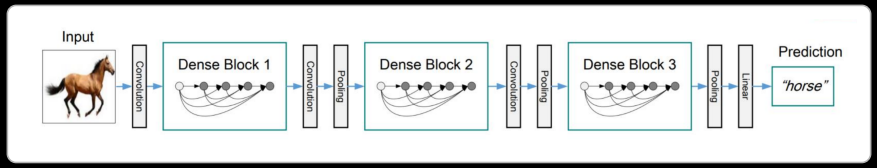
DenseNet (2017): connect more!



Inside one dense block: every layer receives the feature maps of *all* preceding layers.

An extension of the reasoning

- DenseNet proposes **entire blocks of layers connected to one another**, which **diversifies** the features within those blocks far more.
- One character of difference:
  ResNet: $x_\ell = \mathcal{F}(x_{\ell-1}) + x_{\ell-1}$ (**add**)
  DenseNet: $x_\ell = \mathcal{F}([x_0, \ldots, x_{\ell-1}])$ (**concatenate**)
- Nothing is ever summed away, so every layer sees the raw earlier features, and the net needs far fewer channels per layer.



Huang et al., *Densely Connected Convolutional Networks*, CVPR

Conclusion

Global trends

1. A major pattern observed overall: networks are designed to be **deeper and deeper**.
2. **Computational tricks** (ReLU, dropout, batch normalization) were introduced alongside them, with a significant impact on performance.
3. The apparition of **modules** able to capture **rich features** at each step of the network.
4. The increasing use of **connections between the layers**, which helps produce diverse features and proved useful for **gradient propagation**.

Next lecture: how to actually train one, and what to do when medical labels are scarce.

Amidi & Amidi, *The evolution of image classification explained*